LUXE - Tài Liệu Luồng Hoạt Động Hệ Thống

> Tài liệu này mô tả chi tiết cách toàn bộ hệ thống LUXE (nền tảng thương mại điện tử xa xỉ) hoạt động — từ giao diện người dùng, qua backend xử lý nghiệp vụ, đến dịch vụ AI tư vấn thời trang. Mỗi luồng đều được giải thích bằng lời kèm sơ đồ trực quan để người đọc có thể hiểu được toàn bộ quy trình vận hành.

1. Tổng Quan Kiến Trúc

Hệ thống LUXE gồm **4 thành phần chính** giao tiếp với nhau qua HTTP REST API:

- **Frontend (React + Vite, port 3000):** Giao diện người dùng chạy trên trình duyệt. Tất cả các trang (Trang chủ, Sản phẩm, Giỏ hàng, Chat AI, Hồ sơ...) đều gọi API đến Backend thông qua thư viện `axios`. Mỗi request được tự động đính kèm JWT token để xác thực.
- **Backend (Spring Boot, port 8080):** Server Java xử lý toàn bộ logic nghiệp vụ — xác thực người dùng (JWT), quản lý sản phẩm, giỏ hàng, đơn hàng, đánh giá, wishlist. Khi cần tư vấn AI, Backend gọi tiếp đến Python AI Service.
- **AI Advisor Service (Python FastAPI, port 8001):** Dịch vụ AI chuyên biệt sử dụng NLP (Sentence Transformers) để phân tích câu hỏi của người dùng, xếp hạng sản phẩm phù hợp, và tạo phản hồi tư vấn thời trang bằng tiếng Việt.
- **PostgreSQL 16 (Docker, port 5432):** Cơ sở dữ liệu lưu trữ tất cả dữ liệu — users, products, orders, cart, reviews, wishlist, chat history.

Ngoài ra còn **pgAdmin 4 (port 5050)** làm công cụ quản lý database qua giao diện web.

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        TRÌNH DUYỆT (Client)                              │
│  React + Vite (port 3000)                                                 │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐      │
│  │   Home   │ │ Products │ │   Cart   │ │ AI Chat  │ │ Profile  │      │
│  └──────────┘ └──────────┘ └──────────┘ └──────────┘ └──────────┘      │
│       └──────────┴──────────┴──────────┴──────────┴──────────┘           │
│                                │                                          │
│                    axios (api.js) + JWT Token                             │
└──────────────────────────────────────────────────────────────────────────┘
                                │
                                │ HTTP (REST API)
                                ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                   BACKEND - Spring Boot (port 8080)                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐                  │
│  │ SecurityConfig│  │ JWT Filter   │  │ Controllers  │                  │
│  │ (CORS + Auth) │  │ (Token Parse)│  │ (REST Endpoints│                  │
│  └──────────────┘  └──────────────┘  └──────────────┘                  │
│        │                  │                  │                           │
│        ▼                  ▼                  ▼                           │
│  ┌──────────────────────────────────────────────┐                       │
│  │              Service Layer                   │                       │
│  │  AuthService │ ProductService │ AiStylistService │                   │
│  └──────────────┴────────────────┴──────────────┘                       │
│                                          │ HTTP POST /advise             │
│  ┌──────────────────┐                    │                               │
│  │  JPA Repositories │                    │                               │
│  └──────────────────┘                    │                               │
└──────────────┬──────────────────────────┴──────────────────────────────┘
               │ JDBC                      │
               ▼                           ▼
┌──────────────────────┐     ┌──────────────────────────────────────┐
│  PostgreSQL 16       │     │  AI Advisor Service - FastAPI (port 8001) │
│  (port 5432)         │     │  ┌──────────────────────────┐        │
│  DB: luxury_ecommerce│     │  │ SalesSemanticAdvisor     │        │
│                      │     │  │ - Sentence Transformers (NLP)  │        │
│  Tables:             │     │  │ - SPIN+FAB Framework     │        │
│  - users             │     │  │ - Product Ranking        │        │
│  - products          │     │  └──────────────────────────┘        │
│  - carts/cart_items  │     └──────────────────────────────────────┘
│  - orders/order_items│
│  - reviews           │     ┌──────────────────────────────────────┐
│  - wishlist          │     │  pgAdmin 4 (port 5050)               │
```

```
│  - chat_history      │     │  Quản lý cơ sở dữ liệu                  │
└──────────────────────┘     └─────────────────────────────────────────┘
```

2. Luồng Xác Thực (Authentication Flow)

Xác thực là nền tảng bảo mật của toàn hệ thống. LUXE sử dụng **JWT (JSON Web Token)** với thuật toán **HS512** và thời hạn **24 giờ**. Khi người dùng đăng nhập thành công, server sinh ra một token chứa thông tin username, thời gian tạo và thời gian hết hạn. Token này được frontend lưu trong `localStorage` và tự động đính kèm vào mọi request API thông qua axios interceptor.

Hệ thống không sử dụng cookie hay session phía server — hoàn toàn **stateless**. Mỗi request đều được xác thực độc lập bằng cách kiểm tra token trong header `Authorization: Bearer <token>` .

2.1. Đăng Ký Tài Khoản

Khi người dùng đăng ký, Frontend gửi thông tin (username, email, password, họ tên) đến Backend. Backend kiểm tra username và email chưa tồn tại trong database, sau đó mã hoá password bằng **BCrypt** (không bao giờ lưu mật khẩu dạng plain text), tạo bản ghi User mới trong database, sinh JWT token và trả về cho Frontend. Frontend lưu token + thông tin user vào `localStorage` rồi chuyển hướng về trang chủ — người dùng được đăng nhập tự động ngay sau khi đăng ký.

```
Người dùng                  Frontend (React)             Backend (Spring Boot)          Datab
    |                              |                              |                          |
    |  Nhập username/email/pass    |                              |                          |
    |  tại trang /register         |                              |                          |
    |──────────────────────────────►|                              |                          |
    |                              |  POST /api/auth/register     |                          |
    |                              |  Body: {username, email,     |                          |
    |                              |         password, firstName, |                          |
    |                              |         lastName}            |                          |
    |                              |──────────────────────────────►|                          |
    |                              |                              |  Check username tồn tại? |
    |                              |                              |──────────────────────────►|
    |                              |                              |◄──────────────────────────|
    |                              |                              |  Check email tồn tại?    |
    |                              |                              |──────────────────────────►|
    |                              |                              |◄──────────────────────────|
    |                              |                              |  BCrypt hash password    |
    |                              |                              |  INSERT INTO users       |
    |                              |                              |──────────────────────────►|
    |                              |                              |◄──────────────────────────|
    |                              |                              |  Generate JWT token      |
    |                              |  200 OK                      |  (HS512, hết hạn 24h)    |
    |                              |  {status: "success",         |                          |
    |                              |   data: {token, id,          |                          |
    |                              |     username, email, role}}  |                          |
    |                              |◄──────────────────────────────|                          |
    |                              |                              |                          |
    |                              |  localStorage.setItem        |                          |
    |                              |    ('token', jwt)            |                          |
    |                              |    ('user', JSON userData)   |                          |
    |  Chuyển hướng → Trang chủ    |                              |                          |
    |◄──────────────────────────────|                              |                          |
```

File liên quan:

- Frontend: `src/services/api.js` → `authAPI.register()`
- Frontend: `src/context/AuthContext.jsx` → `register()` cập nhật user state
- Backend: `AuthController.java` → `POST /api/auth/register`
- Backend: `AuthService.java` → Hash password, tạo User entity, sinh JWT

2.2. Đăng Nhập

Khi đăng nhập, Backend sử dụng `AuthenticationManager` của Spring Security để xác thực. Quy trình bên trong: `CustomUserDetailsService` tải user từ database theo username, Spring Security so sánh password người dùng nhập với hash BCrypt đã lưu. Nếu khớp → sinh JWT token mới và trả về. Nếu sai → trả lỗi 400. Frontend nhận token, lưu vào `localStorage` (key `token` và `user`), cập nhật AuthContext state, và chuyển hướng về trang chủ.

```
Người dùng                  Frontend (React)             Backend (Spring Boot)          Datab
    |                              |                              |                          |
    |  Nhập username + password    |                              |                          |
    |  tại trang /login            |                              |                          |
    |──────────────────────────────►|                              |                          |
    |                              |  POST /api/auth/login        |                          |
    |                              |  Body: {username, password}  |                          |
    |                              |──────────────────────────────►|                          |
    |                              |                              |                          |
    |                              |                              |  AuthenticationManager   |
    |                              |                              |  .authenticate()         |
    |                              |                              |     |                    |
    |                              |                              |     ▼                    |
    |                              |                              |  CustomUserDetailsService |
    |                              |                              |  .loadByUsername()       |
    |                              |                              |──────────────────────────►|
    |                              |                              |  SELECT * FROM users     |
    |                              |                              |  WHERE username = ?      |
    |                              |                              |◄──────────────────────────|
    |                              |                              |                          |
    |                              |                              |  BCrypt.matches(         |
    |                              |                              |    plainPass, hashPass)  |
    |                              |                              |                          |
    |                              |                              |  Nếu ĐÚNG:               |
    |                              |                              |  → Sinh JWT token (HS512) |
    |                              |                              |  → Trả {token, user info} |
    |                              |                              |                          |
    |                              |                              |  Nếu SAI:                |
    |                              |                              |  → 400 "Invalid username  |
    |                              |                              |    or password"
    |                              |                              |                          |
    |                              |  200 OK                      |                          |
    |                              |  {token, id, username,       |                          |
    |                              |   email, role}               |                          |
    |                              |◄──────────────────────────────|                          |
    |                              |                              |                          |
    |                              |  localStorage.setItem(       |                          |
    |                              |    'token', jwt)             |                          |
    |                              |  localStorage.setItem(       |                          |
    |                              |    'user', JSON userData)    |                          |
    |  Chuyển hướng → Trang chủ   |                              |                          |
    |◄──────────────────────────────|                              |                          |
```

2.3. Xác Thực Từng Request (JWT Filter)

Mọi request đến Backend đều đi qua `JwtAuthenticationFilter` — một filter trong chuỗi Spring Security filter chain. Filter này hoạt động như "người gác cổng": nó trích xuất token từ header `Authorization`, kiểm tra chữ ký số (đảm bảo token không bị giả mạo) và thời hạn (đảm bảo chưa hết hạn), rồi tải thông tin user từ database và đặt vào `SecurityContext`. Sau đó, `SecurityConfig` kiểm tra URL pattern để quyết định endpoint này cần quyền gì (public, authenticated, hay admin). Nếu token thiếu hoặc không hợp lệ, `SecurityContext` sẽ rỗng và server trả về **401 Unauthorized**.

```
Trình duyệt                    JwtAuthenticationFilter        SecurityConfig           Control
    │                                  │                           │                        │
    │  Request + Header:               │                           │                        │
    │  Authorization: Bearer <token>   │                           │                        │
    ├──────────────────────────────────►│                           │                        │
    │                                  │                           │                        │
    │                                  │  1. parseJwt(request)     │                        │
    │                                  │     Lấy token từ header   │                        │
    │                                  │     "Authorization"       │                        │
    │                                  │     (bỏ tiền tố "Bearer ")│                        │
    │                                  │                           │                        │
    │                                  │  2. validateToken(jwt)    │                        │
    │                                  │     Check chữ ký HS512    │                        │
    │                                  │     Check hết hạn         │                        │
    │                                  │                           │                        │
    │                                  │  3. extractUsername(jwt)   │
    │                                  │     Lấy "sub" từ payload  │                        │
    │                                  │                           │                        │
    │                                  │  4. loadUserByUsername()   │
    │                                  │     Tải UserDetails từ DB │                        │
    │                                  │                           │                        │
    │                                  │  5. Set SecurityContext   │                        │
    │                                  │     (Authentication obj)  │                        │
    │                                  │                           │                        │
    │                                  │  filterChain.doFilter()  │                        │
    │                                  ├──────────────────────────►│                        │
    │                                  │                           │  Check authorization   │
    │                                  │                           │  theo URL pattern      │
    │                                  │                           ├──────────────────────►│
    │                                  │                           │       ✓ OK            │
    │◄──────────────────────────────────┤◄──────────────────────────┤◄──────────────────────┤
    │                                  │                           │                        │
    │  Nếu token thiếu/hết hạn:        │                           │                        │
    │  → SecurityContext rỗng          │                           │                        │
    │  → 401 Unauthorized              │                           │                        │
    │◄──────────────────────────────────┤                           │                        │
```

2.4. Xử Lý Khi Token Hết Hạn (Frontend Auto-Redirect)

JWT token có thời hạn 24 giờ. Sau khi hết hạn, mọi request sẽ bị Backend từ chối với status **401**. Để tránh trải nghiệm người dùng bị stuck tại trang trắng hoặc lỗi khó hiểu, Frontend có một **axios response interceptor** tự động xử lý: khi nhận được response 401 hoặc 403 (trừ các API công khai

như login/register), interceptor sẽ xoá token và user khỏi `localStorage`, rồi redirect người dùng về trang `/login`. Người dùng chỉ cần đăng nhập lại để lấy token mới.

```
Trình duyệt                    axios response interceptor      Backend
    │                                │                            │
    │  Request với token hết hạn     │                            │
    ├──────────────────────────────►│                            │
    │                                │  Gọi API bất kỳ            │
    │                                ├──────────────────────────►│
    │                                │  ◄── 401 Unauthorized      │
    │                                │◄──────────────────────────┤
    │                                │                            │
    │                                │  Response Interceptor:     │
    │                                │  1. Nhận status 401        │
    │                                │  2. Xóa localStorage       │
    │                                │     (token + user)         │
    │                                │  3. window.location.href   │
    │                                │     = '/login'             │
    │  Tự động về trang Login        │                            │
    │◄──────────────────────────────┤                            │
```

3. Phân Quyền Endpoint (Security Rules)

Spring Security trong `SecurityConfig.java` định nghĩa rõ từng nhóm URL được phép truy cập bởi ai. Hệ thống chia làm 3 cấp: **Public** (ai cũng xem được, như danh sách sản phẩm), **Authenticated** (cần đăng nhập, như giỏ hàng/đặt hàng), và **Admin** (chỉ tài khoản có role `ADMIN` mới truy cập được, như quản lý sản phẩm/users). Ngoài CORS được cấu hình cho phép frontend gọi từ port 3000/5173, tất cả request đều phải qua JWT filter trước khi đến controller.

SecurityConfig.java định nghĩa:

Pattern	Quyền truy cập	Giải thích
`/api/auth/**`	`permitAll`	Đăng nhập/đăng ký không cần token
`/api/products/**`	`permitAll`	Xem sản phẩm không cần đăng nhập

Pattern	Quyền truy cập	Giải thích
`/api/reviews/product/**`	`permitAll`	Xem review sản phẩm công khai
`/api/orders/**`	`authenticated`	Cần đăng nhập để đặt/xem đơn hàng
`/api/cart/**`	`authenticated`	Cần đăng nhập để dùng giỏ hàng
`/api/ai-stylist/**`	`authenticated`	Cần đăng nhập để chat AI
`/api/wishlist/**`	`authenticated`	Cần đăng nhập để yêu thích
`/api/users/**`	`authenticated`	Cần đăng nhập để xem hồ sơ
`/api/admin/**`	`authenticated` + `ROLE_ADMIN`	Chỉ admin
Tất cả còn lại	`authenticated`	Mặc định cần đăng nhập

4. Luồng Sản Phẩm (Product Flow)

Sản phẩm là trung tâm của hệ thống. Người dùng có thể xem danh sách và chi tiết sản phẩm **mà không cần đăng nhập** — đây là thiết kế phổ biến trong thương mại điện tử để thu hút khách truy cập. Dữ liệu sản phẩm bao gồm: tên, giá, ảnh, mô tả, danh mục (Timepieces, Jewelry, Accessories...), thương hiệu, màu sắc, chất liệu, trạng thái premium, và số lượng tồn kho.

4.1. Xem Danh Sách Sản Phẩm (Không cần đăng nhập)

Khi người dùng mở trang `/products`, component `ProductList.jsx` trong `useEffect()` gọi `GET /api/products` để tải toàn bộ sản phẩm. Backend trả về danh sách đầy đủ, sau đó **Frontend xử lý lọc/sắp xếp/phân trang hoàn toàn phía client** (không gọi lại API mỗi khi filter thay đổi). Việc này giúp tốc độ lọc/tìm kiếm rất nhanh vì không cần chờ network.

```
Người dùng                  ProductList.jsx              Backend                        Databa
    │                            │                          │                              │
    │  Truy cập /products        │                          │                              │
    ├──────────────────────────►│                          │                              │
    │                            │  useEffect() khi mount   │                              │
    │                            │                          │                              │
    │                            │  GET /api/products       │                              │
    │                            │  (không cần token)       │                              │
    │                            ├──────────────────────────►│                              │
    │                            │                          │  ProductController           │
    │                            │                          │  .getAllProducts()
    │                            │                          │  → productRepository         │
    │                            │                          │    .findAll()                │
    │                            │                          ├──────────────────────────────►│
    │                            │                          │  SELECT * FROM products      │
    │                            │                          ◄──────────────────────────────┤
    │                            │                          │                              │
    │                            │  {status: "success",     │                              │
    │                            │   data: [{id, name,      │                              │
    │                            │     price, imageUrl,     │                              │
    │                            │     category, isPremium, │                              │
    │                            │     stockQuantity, brand,│                              │
    │                            │     color, material}]}   │                              │
    │                            ◄──────────────────────────┤                              │
    │                            │                          │                              │
    │  Hiển thị grid sản phẩm    │  Frontend xử lý local:   │                              │
    │  với các chức năng:        │  - Lọc theo category     │                              │
    │  ✓ Tìm kiếm (search)       │  - Lọc theo premium      │                              │
    │  ✓ Lọc danh mục            │  - Sắp xếp theo giá/tên  │                              │
    │  ✓ Sắp xếp                 │  - Phân trang (12 SP/trang)│                            │
    │  ✓ Phân trang              │  - Match keyword search  │                              │
    ◄──────────────────────────┤                          │                              │
```

4.2. Xem Chi Tiết Sản Phẩm

Khi click vào một sản phẩm, người dùng được chuyển đến trang `/products/:id`. Component `ProductDetail.jsx` gọi **song song** 2-3 API cùng lúc để tối ưu tốc độ tải: (1) Lấy chi tiết sản phẩm, (2) Lấy danh sách reviews, và (3) Nếu đã đăng nhập — kiểm tra sản phẩm này có trong wishlist không. Gọi song song giúp trang load nhanh hơn so với gọi tuần tự.

```
Người dùng                  ProductDetail.jsx            Backend                        Databa
    |                            |                           |                              |
    |  Click sản phẩm bất kỳ     |                           |                              |
    |  → URL: /products/:id      |                           |                              |
    |──────────────────────────►|                           |                              |
    |                            |                           |                              |
    |                            | ── Gọi song song ────────|                              |
    |                            | |                         |                              |
    |                            | ├─ GET /api/products/{id} |                              |
    |                            | |  (public, ko cần token) |──────────────────────────►|
    |                            | |                         |  SELECT * FROM products      |
    |                            | |                         |  WHERE id = ?                |
    |                            | |                         |◄──────────────────────────|
    |                            | |                         |                              |
    |                            | ├─ GET /api/reviews/      |                              |
    |                            | |  product/{productId}    |                              |
    |                            | |  (public, ko cần token) |──────────────────────────►|
    |                            | |                         |  SELECT r.*, u.username      |
    |                            | |                         |  FROM reviews r
    |                            | |                         |  JOIN users u
    |                            | |                         |  WHERE r.product_id = ?
    |                            | |                         |◄──────────────────────────|
    |                            | |                         |                              |
    |                            | └─ (Nếu đã đăng nhập)    |                              |
    |                            |    GET /api/wishlist/     |                              |
    |                            |    user/{uid}/product/    |                              |
    |                            |    {pid}/check            |                              |
    |                            |    Header: Bearer <token> |──────────────────────────►|
    |                            |                           |  SELECT * FROM wishlist      |
    |                            |                           |  WHERE user_id=? AND         |
    |                            |                           |  product_id=?                |
    |                            |                           |◄──────────────────────────|
    |                            |                           |                              |
    |  Hiển thị:                 |                           |                              |
    |  - Ảnh sản phẩm lớn        |                           |                              |
    |  - Tên, giá (VNĐ)          |                           |                              |
    |  - Mô tả chi tiết          |                           |                              |
    |  - Thuộc tính (màu, chất   |                           |                              |
    |    liệu, thương hiệu)      |                           |                              |
    |  - Danh sách reviews +     |                           |                              |
    |    rating trung bình       |                           |                              |
    |  - Nút "Thêm vào giỏ"      |                           |                              |
    |  - Nút ♥ yêu thích         |                           |                              |
```

```
│  - Breadcrumb navigation    │                          │                          │
│◄──────────────────────────────┤                          │                          │
```

5. Luồng Giỏ Hàng (Cart Flow)

Giỏ hàng được quản lý **cả phía server (database) lẫn phía client (React Context)**. Mỗi user có tối đa một giỏ hàng (`carts` table), chứa nhiều `cart_items` . `CartContext` ở frontend đóng vai trò cache — lưu trạng thái giỏ hàng hiện tại trong React state để các component (Navbar badge, Cart page, Checkout) có thể truy cập mà không cần gọi lại API. Khi user thêm/sửa/xoá item, `CartContext` gọi API đến Backend rồi cập nhật state local, đảm bảo UI luôn đồng bộ với database.

5.1. Thêm Sản Phẩm Vào Giỏ

Người dùng có thể thêm sản phẩm vào giỏ từ bất kỳ trang nào có nút "Thêm vào giỏ" (trang danh sách, chi tiết, hoặc kết quả AI). Khi click, component gọi `addToCart()` từ `CartContext` . CartContext gửi `POST /api/cart/user/{userId}/items` kèm JWT token. Backend tìm (hoặc tạo mới) giỏ hàng của user, thêm item vào `cart_items` , rồi trả về cart đã cập nhật. Frontend cập nhật state → Navbar badge tự động hiển thị số lượng mới.

```
Người dùng                  Component (bất kỳ)           CartContext              Backend
    |                            |                            |                        |
    |  Click "Thêm vào giỏ"      |                            |                        |
    |----------------------------|                            |                        |
    |                            |  addToCart(product, qty)   |                        |
    |                            |----------------------------|                        |
    |                            |                            |  POST /api/cart/       |
    |                            |                            |  user/{userId}/items   |
    |                            |                            |  Body: {productId,     |
    |                            |                            |         quantity: 1}   |
    |                            |                            |  Header: Bearer <token>|
    |                            |                            |------------------------|
    |                            |                            |                        |  CartC
    |                            |                            |                        |  .addI
    |                            |                            |                        |
    |                            |                            |                        |  1. Ti
    |                            |                            |                        |     ch
    |                            |                            |                        |--------
    |                            |                            |                        |  SELEC
    |                            |                            |                        |<-------
    |                            |                            |                        |
    |                            |                            |                        |  2. Th
    |                            |                            |                        |--------
    |                            |                            |                        |  INSER
    |                            |                            |                        |  (cart
    |                            |                            |                        |   quan
    |                            |                            |                        |<-------
    |                            |                            |                        |
    |                            |                            |  {cart object +        |
    |                            |                            |   updated items}       |
    |                            |                            |<-----------------------|
    |                            |                            |                        |
    |                            |                            |  Cập nhật cart state   |
    |                            |<---------------------------|  (Context re-render)   |
    |                            |                            |                        |
    |  Navbar badge số lượng     |                            |                        |
    |  cập nhật tự động          |                            |                        |
    |<---------------------------|                            |                        |
```

5.2. Xem & Chỉnh Sửa Giỏ Hàng

Trang `/cart` hiển thị toàn bộ sản phẩm trong giỏ với ảnh, tên, giá, số lượng (có thể chỉnh +/−) và tổng tiền. Mỗi thay đổi số lượng gọi `PUT` để cập nhật, mỗi lần xoá gọi `DELETE`. Backend JOIN bảng `carts` → `cart_items` → `products` để trả về đầy đủ thông tin sản phẩm trong giỏ.

```
Người dùng                    Cart.jsx                      Backend                        Databa
    │                             │                            │                              │
    │  Truy cập /cart             │                            │                              │
    ├──────────────────────────────►│                            │                              │
    │                             │  GET /api/cart/            │
    │                             │  user/{userId}             │                              │
    │                             ├────────────────────────────►│                              │
    │                             │  {cart: {items: [          │  SELECT c.*, ci.*, p.*       │
    │                             │    {product, qty, price}   │  FROM carts c                │
    │                             │  ]}}                       │  JOIN cart_items ci
    │                             │◄────────────────────────────┤  JOIN products p
    │                             │                            │  WHERE c.user_id = ?
    │  Hiển thị giỏ hàng          │                            │                              │
    │  - Danh sách sản phẩm       │                            │                              │
    │  - Số lượng (có thể sửa)    │                            │                              │
    │  - Tổng tiền                │                            │                              │
    │◄──────────────────────────────┤                            │                              │
    │                             │                            │                              │
    │  Thay đổi số lượng (+/-)    │  PUT /api/cart/            │                              │
    │                             │  user/{uid}/items/{itemId}│                              │
    │                             │  Body: {quantity: n}       │  UPDATE cart_items
    │                             ├────────────────────────────►│  SET quantity = ?
    │                             │◄────────────────────────────┤  WHERE id = ?                │
    │                             │                            │                              │
    │  Xóa sản phẩm (×)           │  DELETE /api/cart/         │                              │
    │                             │  user/{uid}/items/{itemId}│  DELETE FROM cart_items
    │                             ├────────────────────────────►│  WHERE id = ?                │
    │                             │◄────────────────────────────┤                              │
```

6. Luồng Đặt Hàng (Order Flow)

Đặt hàng là bước chuyển từ "có ý định mua" sang "đã mua". Quy trình gồm 2 bước: (1) Checkout — nhập thông tin giao hàng, (2) Tạo đơn hàng — Backend tạo order, trừ kho, xoá giỏ. Đơn hàng có 5

trạng thái: PENDING → CONFIRMED → SHIPPED → DELIVERED / CANCELLED. Admin có thể cập nhật trạng thái qua API riêng.

6.1. Thanh Toán (Checkout → Order)

Từ trang giỏ hàng, người dùng click "Tiến hành thanh toán" → chuyển đến `/checkout`. Trang checkout hiển thị tóm tắt giỏ hàng kèm form nhập tên người nhận, số điện thoại, địa chỉ giao hàng. Khi xác nhận, Frontend gửi `POST /api/orders` kèm danh sách items. Backend thực hiện **4 bước trong cùng một transaction**: (1) Tạo bản ghi order với trạng thái PENDING, (2) Tạo các order_items, (3) Trừ `stock_quantity` của từng sản phẩm, (4) Xoá toàn bộ cart_items. Frontend sau đó clear CartContext và chuyển đến trang "Đặt hàng thành công".

```
Người dùng                  Checkout.jsx                  Backend                        Databa
    |                            |                            |                              |
    |  Từ /cart click            |                            |                              |
    |  "Tiến hành thanh toán"    |                            |                              |
    |  → Chuyến đến /checkout    |                            |                              |
    |──────────────────────────►|                            |                              |
    |                            |                            |                              |
    |  Hiển thị form:            |                            |                              |
    |  - Tóm tắt giỏ hàng       |                           |                              |
    |  - Nhập tên người nhận     |                            |                              |
    |  - Nhập số điện thoại      |                            |                              |
    |  - Nhập địa chỉ giao hàng  |                           |                              |
    |                            |                            |                              |
    |  Click "Xác nhận đặt hàng" |                           |                              |
    |──────────────────────────►|                            |                              |
    |                            |  POST /api/orders          |                              |
    |                            |  Header: Bearer <token>    |                              |
    |                            |  Body: {                   |                              |
    |                            |    userId,                 |                              |
    |                            |    shippingAddress,        |                              |
    |                            |    phoneNumber,            |                              |
    |                            |    items: [                |                              |
    |                            |      {productId: 1,        |                              |
    |                            |       quantity: 2},        |                              |
    |                            |      {productId: 3,        |                              |
    |                            |       quantity: 1}         |                              |
    |                            |    ]                       |                              |
    |                            |  }                         |                              |
    |                            |──────────────────────────►|                              |
    |                            |                            |  OrderController             |
    |                            |                            |  .createOrder()              |
    |                            |                            |                              |
    |                            |                            |  1. Tạo Order (PENDING)     |
    |                            |                            |──────────────────────────────►|
    |                            |                            |  INSERT INTO orders          |
    |                            |                            |  (user_id, total_amount,     |
    |                            |                            |   status='PENDING',          |
    |                            |                            |   shipping_address, phone)   |
    |                            |                            |◄──────────────────────────────|
    |                            |                            |                              |
    |                            |                            |  2. Tạo OrderItems           |
    |                            |                            |──────────────────────────────►|
    |                            |                            |  INSERT INTO order_items     |
```

```
│                            │                           │  (order_id, product_id,      │
│                            │                           │   quantity, price)           │
│                            │                           │◄─────────────────────────────┤
│                            │                           │                              │
│                            │                           │  3. Trừ tồn kho              │
│                            │                           ├─────────────────────────────►│
│                            │                           │  UPDATE products             │
│                            │                           │  SET stock_quantity =        │
│                            │                           │  stock_quantity - ?          │
│                            │                           │◄─────────────────────────────┤
│                            │                           │                              │
│                            │                           │  4. Xóa giỏ hàng             │
│                            │                           ├─────────────────────────────►│
│                            │                           │  DELETE FROM cart_items      │
│                            │                           │  WHERE cart_id = ?           │
│                            │                           │◄─────────────────────────────┤
│                            │                           │                              │
│                            │  {order object với        │                              │
│                            │   id, items, total}       │                              │
│                            │◄──────────────────────────┤                              │
│                            │                           │                              │
│                            │  CartContext.clearCart()  │
│                            │  navigate('/order-success')│
│                            │                           │                              │
│  Trang "Đặt hàng thành     │                           │                              │
│  công!" hiển thị           │                           │                              │
│◄───────────────────────────┤                           │                              │
```

6.2. Xem Lịch Sử Đơn Hàng

Trang `/orders` hiển thị tất cả đơn hàng của user, sắp xếp theo thời gian mới nhất. Mỗi đơn hiện mã đơn, ngày đặt, tổng tiền (format VNĐ), danh sách sản phẩm, và trạng thái hiện tại (Chờ xác nhận / Đã xác nhận / Đang giao / Đã giao / Đã huỷ) với badge màu tương ứng.

```
Người dùng                  OrderHistory.jsx             Backend                        Databa
    |                              |                          |                              |
    |  Truy cập /orders            |                          |                              |
    |──────────────────────────────►|                          |                              |
    |                              |  GET /api/orders/         |
    |                              |  user/{userId}           |                              |
    |                              |  Header: Bearer <token>  |                              |
    |                              |──────────────────────────►|                              |
    |                              |                          |  SELECT o.*, oi.*, p.name    |
    |                              |                          |  FROM orders o               |
    |                              |                          |  JOIN order_items oi
    |                              |                          |  JOIN products p
    |                              |                          |  WHERE o.user_id = ?
    |                              |                          |  ORDER BY o.created_at DESC  |
    |                              |                          |──────────────────────────────►|
    |                              |                          |◄──────────────────────────────|
    |                              |                          |                              |
    |                              |  [{id, status, total,    |                              |
    |                              |    items, createdAt}]    |                              |
    |                              |◄──────────────────────────|                              |
    |                              |                          |                              |
    |  Hiển thị danh sách đơn:    |                          |                              |
    |  - Mã đơn hàng              |                          |                              |
    |  - Ngày đặt                 |                          |                              |
    |  - Tổng tiền (VNĐ)          |                          |                              |
    |  - Trạng thái:              |                          |                              |
    |    PENDING → Chờ xác nhận   |                          |                              |
    |    CONFIRMED → Đã xác nhận  |                          |                              |
    |    SHIPPED → Đang giao      |                          |                              |
    |    DELIVERED → Đã giao      |                          |                              |
    |    CANCELLED → Đã huỷ       |                          |                              |
    |◄────────────────────────────|                          |                              |
```

7. Luồng AI Stylist (Tư Vấn Thời Trang AI)

Đây là luồng phức tạp nhất trong toàn hệ thống, liên kết cả 3 service: Frontend → Backend → AI Python.

Ý tưởng: Người dùng nhắn tin (bằng tiếng Việt) mô tả nhu cầu thời trang — ví dụ "Tư vấn outfit dự tiệc ngân sách 5 triệu" — và AI sẽ phân tích câu hỏi, xếp hạng sản phẩm phù hợp nhất, rồi trả lời bằng tiếng Việt kèm danh sách sản phẩm gợi ý có thể mua ngay. AI sử dụng **6 yếu tố chấm điểm**: keyword matching, ý định (party/work/casual), sở thích cá nhân user, ngân sách, NLP semantic similarity, và tồn kho. Kết hợp lại tạo ra hệ thống gợi ý thông minh, cá nhân hoá cho từng người dùng.

7.1. Mở Trang AI Stylist

Khi truy cập `/ai-stylist`, Frontend đầu tiên kiểm tra đăng nhập — nếu chưa sẽ redirect về `/login`. Nếu đã đăng nhập, Frontend gọi `GET /api/ai-stylist/history` để tải lịch sử chat trước đó (nếu có). Backend lấy userId từ JWT token rồi query bảng `chat_history` theo user. Lịch sử được hiển thị dưới dạng chat bubbles giống ứng dụng nhắn tin: bubble trắng cho tin nhắn user, bubble có màu cho phản hồi AI.

```
Người dùng                  AiStylist.jsx                Backend                        Databa
    |                            |                           |                              |
    |  Truy cập /ai-stylist      |                           |                              |
    |──────────────────────────►|                           |                              |
    |                            |                           |                              |
    |                            |  1. Check đăng nhập:      |                              |
    |                            |  authAPI.getCurrentUser() |                              |
    |                            |  → Đọc localStorage       |                              |
    |                            |  → Nếu null → redirect    |                              |
    |                            |    /login                 |                              |
    |                            |                           |                              |
    |                            |  2. Load lịch sử chat:    |                              |
    |                            |  GET /api/ai-stylist/     |                              |
    |                            |  history                  |                              |
    |                            |  Header: Bearer <token>   |                              |
    |                            |──────────────────────────►|                              |
    |                            |                           |  AiStylistController         |
    |                            |                           |  .getCurrentUserHistory()    |
    |                            |                           |  → Lấy userId từ JWT        |
    |                            |                           |                              |
    |                            |                           |  SELECT * FROM               |
    |                            |                           |  chat_history                |
    |                            |                           |  WHERE user_id = ?           |
    |                            |                           |  ORDER BY created_at ASC     |
    |                            |                           |──────────────────────────►|
    |                            |                           |◄──────────────────────────|
    |                            |                           |                              |
    |                            |  [{id, prompt, response,  |                              |
    |                            |    createdAt}]            |                              |
    |                            |◄──────────────────────────|                              |
    |                            |                           |                              |
    |  Hiển thị giao diện chat   |  Format lịch sử thành     |                              |
    |  với lịch sử tin nhắn      |  bubbles (user + ai)      |                              |
    |◄──────────────────────────|                           |                              |
```

7.2. Gửi Tin Nhắn Chat — Luồng Đầy Đủ 3 Service

Đây là luồng chính của AI Stylist, đi qua **10 bước** xuyên suốt cả 3 service:

Phía Frontend (bước UI): Khi user nhấn Gửi, Frontend ngay lập tức thêm bubble tin nhắn user, hiện loading spinner, xoá ô input, và scroll xuống cuối — tạo cảm giác phản hồi tức thì. Đồng thời gửi `POST /api/ai-stylist/chat` kèm prompt.

Phía Backend (bước ①–⑥): Server xác thực JWT, lấy userId, rồi tải **3 nguồn dữ liệu** từ database: (1) Profile user (phong cách, màu yêu thích — cho cá nhân hoá), (2) Toàn bộ sản phẩm (cho AI xếp hạng), (3) 6 tin nhắn chat gần nhất (cho AI hiểu ngữ cảnh trước đó). Sau đó Backend đóng gói tất cả thành JSON và gọi `POST http://localhost:8001/advise` đến Python AI Service.

Phía AI Python (bước chi tiết ở mục 7.3): AI nhận request, phân tích ngôn ngữ tự nhiên, chấm điểm từng sản phẩm, chọn top phù hợp nhất, viết phản hồi tiếng Việt.

Quay về Backend (bước ⑦–⑨): Nhận kết quả AI, map product IDs thành objects đầy đủ, lưu vào `chat_history`, trả response cho Frontend.

Quay về Frontend (bước ⑩): Hiển thị bubble AI response + product cards bên dưới, tắt loading.

```
Người dùng                  AiStylist.jsx                Backend (AiStylistService)    AI Pytho
    |                            |                            |                              |
    |  Gõ: "Tư vấn outfit dự     |                            |                              |
    |  tiệc ngân sách 5 triệu"   |                            |                              |
    |  Nhấn Enter / Click Gửi    |                            |                              |
    |──────────────────────────►|                            |                              |
    |                            |                            |                              |
    |                            |  Bước UI:                  |                              |
    |                            |  1. Thêm bubble user msg   |                              |
    |                            |  2. Hiện loading spinner   |                              |
    |                            |  3. Xoá input box          |                              |
    |                            |  4. Scroll xuống cuối      |                              |
    |                            |                            |                              |
    |                            |  POST /api/ai-stylist/chat|                              |
    |                            |  Header: Bearer <token>    |                              |
    |                            |  Body: {prompt:            |
    |                            |    "Tư vấn outfit dự       |                              |
    |                            |     tiệc ngân sách         |                              |
    |                            |     5 triệu"}              |                              |
    |                            |──────────────────────────►|                              |
    |                            |                            |                              |
    |                            |                            | ① JwtFilter xác thực token   |
    |                            |                            | ② resolveAuthenticatedUserId |
    |                            |                            |   → username từ JWT          |
    |                            |                            |   → tìm User entity          |
    |                            |                            |                              |
    |                            |                            | ③ Tải User profile           |
    |                            |                            |──────────────────────────────┤
    |                            |                            |  SELECT * FROM users
    |                            |                            |  WHERE id = ?                |
    |                            |                            |  (lấy style, favoriteColor)  |
    |                            |                            |◄─────────────────────────────┤
    |                            |                            |                              |
    |                            |                            | ④ Tải tất cả sản phẩm        |
    |                            |                            |──────────────────────────────┤
    |                            |                            |  SELECT * FROM products      |
    |                            |                            |  (8 sản phẩm hiện tại)       |
    |                            |                            |◄─────────────────────────────┤
    |                            |                            |                              |
    |                            |                            | ⑤ Tải 6 chat lịch sử gần     |
    |                            |                            |   nhất của user              |
    |                            |                            |──────────────────────────────┤
    |                            |                            |  SELECT * FROM chat_history  |
```

```
│                          │                          │  WHERE user_id = ?          │
│                          │                          │  ORDER BY created_at DESC   │
│                          │                          │  LIMIT 6                    │
│                          │                          │◄────────────────────────────┤
│                          │                          │                             │
│                          │                          │ ⑥ Gọi AI Python Service    │
│                          │                          │ POST http://localhost:8001
│                          │                          │      /advise                │
│                          │                          │ Body: {                     │
│                          │                          │   prompt: "Tư vấn...",      │
│                          │                          │   user: {                   │
│                          │                          │     id: 1,                  │
│                          │                          │     username: "admin",
│                          │                          │     gender: null,           │
│                          │                          │     style: null,            │
│                          │                          │     favorite_color: null    │
│                          │                          │   },                        │
│                          │                          │   products: [               │
│                          │                          │     {id:1, name:"Diamond    │
│                          │                          │      Watch", price:12500,   │
│                          │                          │      category:"Timepieces", │
│                          │                          │      is_premium:true, ...},│
│                          │                          │     ... (tất cả sản phẩm)  │
│                          │                          │   ],                        │
│                          │                          │   history: [                │
│                          │                          │     {prompt:"...",
│                          │                          │      response:"..."}, ...  │
│                          │                          │   ]                         │
│                          │                          │ }                           │
│                          │                          ├────────────────────────────►│
│                          │                          │                             │
│                          │                          │                             │
```

7.3. AI Python Service Xử Lý (Chi Tiết Thuật Toán)

Đây là "bộ não" của hệ thống, chạy trong class `SalesSemanticAdvisor`. Khi nhận request, AI thực hiện **6 bước xử lý**:

1. **Normalize text (①):** Bỏ dấu tiếng Việt để dễ so khớp ("Tư vấn" → "tu van"). Dùng `unicodedata` để chuyển về dạng ASCII.
2. **Extract budget (②):** Dùng regex tìm số tiền trong câu. Hỗ trợ các đơn vị: "triệu", "tr", "m", "k", "nghìn". Ví dụ: "5 triệu" → 5,000,000 VNĐ.

3. **Detect intent (③):** Phân loại ý định người dùng dựa trên keyword: `party` (tiệc, dạ hội, cưới), `work` (công sở, họp, văn phòng), `summer` (hè, biển), `casual` (thường ngày, dạo phố), `budget` (rẻ, tiết kiệm), `general` (mặc định).
4. **Check greeting (④):** Nếu câu chỉ là lời chào ("xin chào", "hi") không kèm hỏi sản phẩm, AI trả welcome message mà không xếp hạng sản phẩm.
5. **Rank products (⑤):** Bước quan trọng nhất — chấm điểm mỗi sản phẩm bằng **5 tiêu chí cộng dồn**: keyword match (+1.2/match), intent bonus (+2.0 đến +3.0 tuỳ ngữ cảnh), user preference (style +2.5, color +1.8), budget filter (trong tầm +3.0, ngoài tầm −2.0), NLP semantic similarity (cosine similarity × 4.0 — trọng số cao nhất), và stock penalty (hết hàng −3.0). Sắp xếp giảm dần, lấy top 6 có score > 0.
6. **Build response (⑥):** Ghép 4 phần thành câu trả lời tiếng Việt: opener theo intent ("Bạn đang cần outfit cho sự kiện trang trọng..."), dòng budget ("Với ngân sách 5 triệu VND..."), personalization, và recommendation count.

```
                    POST /advise nhận được request
                              |
                    ┌──────────▼──────────┐
                    | SalesSemanticAdvisor |
                    |      .advise()       |
                    └──────────┬──────────┘
                              |
          ┌──────────────────┼──────────────────┐
          |                  |                  |
┌──────────▼──────────┐  ┌──▼──────────────┐  ┌──▼──────────┐
| ① Normalize text    |  | ② Extract budget |  | ③ Detect intent|
|                     |  |                  |  |               |
| Bỏ dấu tiếng Việt   |  | Regex pattern:   |  | Keyword match: |
| "Tư vấn outfit dự   |  | (\d+)\s*(triệu   |  |               |
|  tiệc ngân sách     |  | |tr|m|k|nghin)   |  | "tiec" chứa   |
|  5 triệu"           |  |                  |  | trong nhóm    |
|        ↓            |  | "5 trieu"        |  | party keywords |
| "tu van outfit du   |  |    ↓             |  |    ↓          |
|  tiec ngan sach     |  | 5 × 1,000,000    |  | intent="party" |
|  5 trieu"           |  | = 5,000,000 VND  |  |               |
└──────────┬──────────┘  └──────┬──────────┘  └──────┬──────────┘
          |                  |                  |
          └──────────────────┼──────────────────┘
                              |
                    ┌──────────▼──────────┐
                    | ④ Check greeting?    |
                    |                      |
                    | Nếu prompt chỉ chào  |
                    | ("xin chao", "hi")   |
                    | VÀ không hỏi sản     |
                    | phẩm → Trả welcome   |
                    | message, KHÔNG xếp   |
                    | hạng sản phẩm        |
                    |                      |
                    | Prompt hiện tại có   |
                    | "tu van" → là product|
                    | query → tiếp tục     |
                    └──────────┬──────────┘
                              |
                    ┌──────────▼──────────┐
                    | ⑤ RANK PRODUCTS      |
                    | (Chấm điểm mỗi SP)   |
                    |                      |
```

```
│ Cho MỖI sản phẩm     │
│ trong danh sách:      │
│                       │
│ score = 0.0           │
│                       │
│ ── Keyword Match ──   │
│ Mỗi token (≥4 ký tự) │
│ trong prompt match    │
│ với text sản phẩm:    │
│   +1.2 điểm/match     │
│                       │
│ ── Intent Bonus ──    │
│ intent="party":       │
│   isPremium → +3.0    │
│   !isPremium → +0.5   │
│ intent="work":        │
│   chứa "blazer/       │
│   briefcase" → +2.5   │
│ intent="casual":      │
│   chứa "wallet/       │
│   accessories" → +2.0 │
│                       │
│ ── User Preference ── │
│ user.style match      │
│   text SP → +2.5      │
│ user.favorite_color   │
│   match → +1.8        │
│                       │
│ ── Budget Filter ──   │
│ price ≤ 5tr → +3.0    │
│ price > 5tr → -2.0    │
│                       │
│ ── NLP Semantic ──    │
│ Sentence Transformers │
│ cosine similarity     │
│ × 4.0 (trọng số cao) │
│ Model: paraphrase-    │
│ multilingual-MiniLM-  │
│ L12-v2                │
│                       │
│ ── Stock Penalty ──   │
│ stock = 0 → -3.0      │
│                       │
```

```
│ Sắp xếp giảm dần      │
│ Lấy top 6 (score > 0)│
└──────────┬──────────┘
           │
┌──────────▼──────────┐
│ ⑥ Build Response     │
│ (Tạo câu trả lời)    │
│                      │
│ Ghép 4 phần:          │
│                      │
│ 1. Opener theo intent:│
│ "Bạn đang cần outfit  │
│  cho sự kiện trang    │
│  trọng — đây là lúc   │
│  để tỏa sáng! ✨"     │
│                      │
│ 2. Budget line:       │
│ "Với ngân sách        │
│  5 triệu VND, mình   │
│  đã lọc ra các SP     │
│  phù hợp nhất."       │
│                      │
│ 3. Personalization:   │
│ "Cập nhật phong cách  │
│  trong hồ sơ để nhận │
│  gợi ý cá nhân hơn!" │
│                      │
│ 4. Recommendation:    │
│ "Mình tìm được 4 sản │
│  phẩm phù hợp — hãy  │
│  xem bên dưới!"       │
└──────────┬──────────┘
           │
           ▼
Return AdvisorResponse {
  response: "Bạn đang cần...",
  recommended_product_ids: [1, 3, 5, 2],
  confidence: 0.78,
  framework: "SPIN+FAB"
}
```

7.4. Backend Nhận Kết Quả AI và Trả Về Frontend

Sau khi Python AI Service trả về kết quả (text response + danh sách product IDs + confidence score), Backend thực hiện 3 bước cuối: **(⑦)** Chuyển đổi danh sách product IDs (ví dụ [1, 3, 5, 2]) thành danh sách Product objects đầy đủ (tên, giá, ảnh, danh mục...) từ dữ liệu đã load trước đó. **(⑧)** Lưu cuộc hội thoại vào bảng `chat_history` (prompt + response + recommended_products). **(⑨)** Trả JSON response chứa cả text lẫn product cards về cho Frontend.

Frontend nhận response xong sẽ **(⑩)** hiển thị bubble AI message (hỗ trợ **bold** formatting) và bên dưới là grid các product cards — mỗi card có ảnh, tên, giá, nút "Thêm vào giỏ" và "Xem chi tiết" để người dùng mua ngay.

```
AI Python Service trả về AdvisorResponse
                            │
                            ▼
      Backend (AiStylistService) tiếp tục:
                            │
              ┌─────────────▼──────────────┐
              │ ⑦ Map product IDs → Object │
              │                            │
              │ recommended_product_ids:    │
              │ [1, 3, 5, 2]               │
              │     ↓                      │
              │ Tìm Product entities       │
              │ từ danh sách đã tải ở ④    │
              │     ↓                      │
              │ List<Product> với đầy đủ   │
              │ thông tin (name, price,     │
              │ imageUrl, ...)             │
              └────────────────────────────┘
                            │
                            │
              ┌─────────────▼──────────────┐
              │ ⑧ Lưu vào chat_history     │   ──────────────────►  Database
              │                            │   INSERT INTO chat_history
              │ user_id = 1                │   (user_id, prompt,
              │ prompt = "Tư vấn outfit..." │    response,
              │ response = "Bạn đang cần..." │    recommended_products)
              │ recommended_products =      │
              │   "1,3,5,2"                │
              └────────────────────────────┘
                            │
                            │
              ┌─────────────▼──────────────┐
              │ ⑨ Trả ChatResponse         │
              │                            │
              │ {                           │
              │   status: "success",        │
              │   data: {                   │
              │     response: "Bạn đang     │
              │       cần outfit cho sự     │
              │       kiện trang trọng...", │
              │     recommendedProducts: [  │
              │       {id:1,                │
              │        name:"Diamond Watch",│
              │        price:12500,          │
              │        imageUrl:"https://...",│
```

```
│        category:"Timepieces",│
│        isPremium:true},      │
│       {id:3, ...},          │
│       {id:5, ...},          │
│       {id:2, ...}           │
│     ]                       │
│   }                         │
│ }                           │
└─────────────────────────────┘
              │
              │
              ▼
Frontend (AiStylist.jsx)
              │
┌─────────────▼──────────────┐
│ ⑩ Hiển thị kết quả         │
│                            │
│ - Bubble AI message:        │
│   Text response có format  │
│   **bold** cho từ khoá      │
│                            │
│ - Product cards bên dưới:   │
│   Ảnh sản phẩm              │
│   Tên + Giá (VNĐ)          │
│   Nút "Thêm vào giỏ"       │
│   Nút "Xem chi tiết"        │
│                            │
│ - Tắt loading spinner       │
│ - Auto-scroll xuống cuối   │
└────────────────────────────┘
```

7.5. Fallback Khi AI Service Không Khả Dụng

Để đảm bảo hệ thống **không bao giờ bị lỗi hoàn toàn** khi AI Python Service gặp sự cố (crash, quá tải, hoặc timeout > 5 giây), Backend có cơ chế **fallback nội bộ** bằng Java. Fallback này sử dụng text matching đơn giản (không có NLP) để tìm sản phẩm theo keyword và tạo phản hồi cơ bản. Chất lượng gợi ý sẽ kém hơn AI, nhưng người dùng vẫn nhận được kết quả thay vì trang lỗi.

```
Nếu AI Python Service bị lỗi hoặc timeout (> 5 giây):

  Backend AiStylistService:
  │
  ├── getPythonRecommendation() trả về null
  │
  ├── Chuyển sang fallback nội bộ (Java):
  │   ├── generateResponse(prompt, user)
  │   │   → Tạo phản hồi đơn giản dựa trên keyword
  │   │
  │   └── getRecommendedProducts(prompt, user, allProducts)
  │       → Lọc sản phẩm bằng text matching đơn giản
  │       → KHÔNG có NLP semantic ranking
  │
  └── Vẫn lưu chat_history và trả kết quả bình thường
      (chất lượng gợi ý thấp hơn nhưng không bị lỗi)
```

7.6. Xóa Lịch Sử Chat

Người dùng có thể xoá toàn bộ lịch sử chat bằng nút "Xóa lịch sử". Frontend gọi `DELETE /api/ai-stylist/history` , Backend xác định user từ JWT rồi xoá tất cả bản ghi trong `chat_history` của user đó. Frontend reset state messages về rỗng.

```
Người dùng                  AiStylist.jsx                Backend                        Databa
    |                            |                           |                              |
    |  Click "Xóa lịch sử"       |                           |                              |
    |──────────────────────────→|                           |                              |
    |                            |  DELETE /api/ai-stylist/  |                              |
    |                            |  history                  |                              |
    |                            |  Header: Bearer <token>   |                              |
    |                            |──────────────────────────→|                              |
    |                            |                           |  resolveAuthenticatedUserId  |
    |                            |                           |  → userId từ JWT             |
    |                            |                           |                              |
    |                            |                           |  DELETE FROM chat_history    |
    |                            |                           |  WHERE user_id = ?           |
    |                            |                           |──────────────────────────→|
    |                            |                           |←──────────────────────────|
    |                            |  {status: "success",      |                              |
    |                            |   message: "cleared"}     |                              |
    |                            |←──────────────────────────|                              |
    |  Chat messages trống       |  setMessages([])          |                              |
    |←──────────────────────────|                           |                              |
```

8. Luồng Wishlist (Danh Sách Yêu Thích)

Wishlist cho phép người dùng đánh dấu sản phẩm yêu thích để xem lại sau. Hoạt động theo kiểu **toggle** (bật/tắt): click icon ♥ lần đầu → thêm vào wishlist (icon chuyển đỏ), click lần nữa → xoá khỏi wishlist (icon trở về trắng). Trang `/wishlist` hiển thị tất cả sản phẩm đã yêu thích. Tại trang chi tiết sản phẩm, khi component mount, Frontend gọi API `check` để biết sản phẩm này đã được yêu thích chưa và hiển thị icon tương ứng.

```
Người dùng                  Frontend                    Backend                     Databa
    |                           |                           |                           |
    |  Click ♥ trên sản phẩm    |                           |                           |
    |---------------------------|                           |                           |
    |                           |  POST /api/wishlist/      |
    |                           |  user/{uid}/product/{pid} |                           |
    |                           |  Header: Bearer <token>   |                           |
    |                           |---------------------------|                           |
    |                           |                           |  WishlistController       |
    |                           |                           |  .addToWishlist()         |
    |                           |                           |  → Check đã tồn tại?      |
    |                           |                           |  → Nếu chưa: INSERT       |
    |                           |                           |---------------------------|
    |                           |                           |  INSERT INTO wishlist     |
    |                           |                           |  (user_id, product_id)    |
    |                           |                           |<--------------------------|
    |                           |                           |                           |
    |                           |  {status: "success"}      |                           |
    |                           |<--------------------------|                           |
    |  Icon ♥ chuyển màu đỏ     |                           |                           |
    |<--------------------------|                           |                           |
    |                           |                           |                           |
    |  Click ♥ lần nữa          |                           |                           |
    |  (bỏ yêu thích)           |                           |                           |
    |---------------------------|                           |                           |
    |                           |  DELETE /api/wishlist/    |
    |                           |  user/{uid}/product/{pid} |                           |
    |                           |---------------------------|                           |
    |                           |                           |  DELETE FROM wishlist     |
    |                           |                           |  WHERE user_id = ?        |
    |                           |                           |  AND product_id = ?       |
    |                           |                           |---------------------------|
    |                           |                           |<--------------------------|
    |  Icon ♥ trở về trắng      |                           |                           |
    |<--------------------------|                           |                           |
```

9. Luồng Đánh Giá Sản Phẩm (Review Flow)

Người dùng đã đăng nhập có thể đánh giá sản phẩm bằng số sao (1-5 ★) và viết bình luận. Reviews hiển thị công khai tại trang chi tiết sản phẩm — **xem review không cần đăng nhập** (endpoint `GET /api/reviews/product/{id}` là public), nhưng **tạo/sửa/xoá review cần đăng nhập**. Khi gửi review mới, trang tự động reload danh sách review để hiện review vừa tạo. Rating trung bình được tính và hiển thị cạnh tên sản phẩm.

```
Người dùng                  ProductDetail.jsx            Backend                        Databa
    |                              |                           |                              |
    |  Tại trang chi tiết SP,      |                           |                              |
    |  nhập đánh giá:             |                          |                              |
    |  - Chọn số sao (1-5 ★)     |                         |                              |
    |  - Viết nội dung review     |                          |                              |
    |  Click "Gửi đánh giá"       |                          |                              |
    |──────────────────────────────►|                           |                              |
    |                              |  POST /api/reviews/        |
    |                              |  user/{userId}            |                              |
    |                              |  Header: Bearer <token>   |                              |
    |                              |  Body: {                  |                              |
    |                              |    productId: 3,          |                              |
    |                              |    rating: 5,             |                              |
    |                              |    comment: "Sản phẩm     |                              |
    |                              |      tuyệt vời!"         |                              |
    |                              |  }                        |                              |
    |                              |──────────────────────────►|                              |
    |                              |                           |  ReviewController            |
    |                              |                           |  .createReview()             |
    |                              |                           |                              |
    |                              |                           |  INSERT INTO reviews         |
    |                              |                           |  (user_id, product_id,       |
    |                              |                           |   rating, comment)           |
    |                              |                           |──────────────────────────────►|
    |                              |                           |◄──────────────────────────────|
    |                              |                           |                              |
    |                              |  {status: "success",      |                              |
    |                              |   data: {review object}}  |                              |
    |                              |◄──────────────────────────|                              |
    |                              |                           |                              |
    |  Review mới xuất hiện       |  Reload danh sách review  |                              |
    |  trong danh sách            |  GET /api/reviews/        |                              |
    |                              |  product/{productId}      |                              |
    |◄──────────────────────────────|                           |                              |
```

10. Bảng Tổng Hợp Tất Cả API Endpoints

Dưới đây là bảng tổng hợp **tất cả 33+ endpoints** của hệ thống, chia theo 4 cấp quyền truy cập. Base URL cho tất cả API: `http://localhost:8080/api` . Mọi endpoint cần xác thực phải gửi kèm header `Authorization: Bearer <jwt_token>` . API internal (Backend → AI) không được expose ra ngoài, chỉ Backend mới gọi được.

10.1. Public (Không cần đăng nhập)

Method	Endpoint	Mô tả	Controller
`POST`	`/api/auth/login`	Đăng nhập, trả JWT token	AuthController
`POST`	`/api/auth/register`	Đăng ký tài khoản	AuthController
`GET`	`/api/products`	Tất cả sản phẩm	ProductController
`GET`	`/api/products/premium`	Sản phẩm cao cấp	ProductController
`GET`	`/api/products/{id}`	Chi tiết 1 sản phẩm	ProductController
`GET`	`/api/products/category/{cat}`	Theo danh mục	ProductController
`GET`	`/api/products/search?keyword=`	Tìm kiếm	ProductController
`GET`	`/api/products/filter?...`	Lọc nâng cao	ProductController
`GET`	`/api/reviews/product/{id}`	Review của sản phẩm	ReviewController

10.2. Authenticated (Cần đăng nhập, gửi kèm JWT)

Method	Endpoint	Mô tả	Controller
`POST`	`/api/ai-stylist/chat`	Chat với AI Stylist	AiStylistController
`GET`	`/api/ai-stylist/history`	Lịch sử chat	AiStylistController
`DELETE`	`/api/ai-stylist/history`	Xóa lịch sử chat	AiStylistController
`GET`	`/api/cart/user/{userId}`	Xem giỏ hàng	CartController

Method	Endpoint	Mô tả	Controller
POST	/api/cart/user/{userId}/items	Thêm SP vào giỏ	CartController
PUT	/api/cart/user/{userId}/items/{itemId}	Sửa số lượng	CartController
DELETE	/api/cart/user/{userId}/items/{itemId}	Xóa SP khỏi giỏ	CartController
DELETE	/api/cart/user/{userId}/clear	Xóa toàn bộ giỏ	CartController
POST	/api/orders	Tạo đơn hàng	OrderController
GET	/api/orders/user/{userId}	Đơn hàng của user	OrderController
GET	/api/orders/{id}	Chi tiết đơn hàng	OrderController
GET	/api/wishlist/user/{userId}	Danh sách yêu thích	WishlistController
POST	/api/wishlist/user/{uid}/product/{pid}	Thêm yêu thích	WishlistController
DELETE	/api/wishlist/user/{uid}/product/{pid}	Bỏ yêu thích	WishlistController
GET	/api/wishlist/user/{uid}/product/{pid}/check	Kiểm tra yêu thích	WishlistController
GET	/api/users/profile	Hồ sơ user hiện tại	UserController
GET	/api/users/{id}	Thông tin user theo ID	UserController
PUT	/api/users/{id}/profile	Cập nhật hồ sơ	UserController
POST	/api/reviews/user/{userId}	Tạo đánh giá	ReviewController

Method	Endpoint	Mô tả	Controller
PUT	/api/reviews/{reviewId}	Sửa đánh giá	ReviewController
DELETE	/api/reviews/{reviewId}	Xóa đánh giá	ReviewController

10.3. Admin Only (Cần role ADMIN)

Method	Endpoint	Mô tả	Controller
GET	/api/orders	Tất cả đơn hàng (admin)	OrderController
PUT	/api/orders/{id}/status	Cập nhật trạng thái đơn	OrderController
POST	/api/products	Thêm sản phẩm mới	ProductController
PUT	/api/products/{id}	Sửa sản phẩm	ProductController
DELETE	/api/products/{id}	Xóa sản phẩm	ProductController
GET	/api/admin/users	Danh sách users	AdminController
PUT	/api/admin/users/{userId}/toggle-vip	Bật/tắt VIP	AdminController

10.4. Internal API (Backend → AI Service, không public)

Method	Endpoint	Mô tả
GET	http://localhost:8001/health	Kiểm tra trạng thái AI service
POST	http://localhost:8001/advise	Gửi yêu cầu tư vấn sản phẩm

11. Cấu Trúc Database

Database PostgreSQL 16 gồm **8 bảng chính** phục vụ toàn bộ nghiệp vụ. Các bảng `users` và `products` là trung tâm, được tham chiếu bởi hầu hết bảng còn lại thông qua foreign key (`user_id` , `product_id`). Đặc biệt, bảng `users` có các trường `style` , `favorite_color` , `gender` dùng cho AI

personalization — khi user cập nhật hồ sơ với sở thích thời trang, AI sẽ gợi ý chính xác hơn. Bảng `chat_history` lưu lại toàn bộ hội thoại AI để user có thể xem lại và AI có ngữ cảnh khi trả lời câu hỏi tiếp theo.

```
-- Bảng người dùng
users (
    id BIGSERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,    -- Tên đăng nhập
    email VARCHAR(100) UNIQUE NOT NULL,      -- Email
    password VARCHAR(255) NOT NULL,          -- BCrypt hash
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    role VARCHAR(20) DEFAULT 'USER',         -- USER hoặc ADMIN
    gender VARCHAR(20),                      -- Cho AI personalization
    style VARCHAR(50),                       -- Phong cách yêu thích
    favorite_color VARCHAR(50),              -- Màu yêu thích
    is_vip BOOLEAN DEFAULT FALSE,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP,
    updated_at TIMESTAMP
)

-- Bảng sản phẩm
products (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    description TEXT,
    price DECIMAL(10,2) NOT NULL,
    image_url VARCHAR(500),
    category VARCHAR(50),                    -- Timepieces, Jewelry, Accessories...
    is_premium BOOLEAN DEFAULT FALSE,
    stock_quantity INTEGER DEFAULT 0,
    brand VARCHAR(100),                      -- Cho AI matching
    color VARCHAR(50),                       -- Cho AI matching
    material VARCHAR(50),                    -- Cho AI matching
    size VARCHAR(20),
    created_at TIMESTAMP,
    updated_at TIMESTAMP
)

-- Giỏ hàng
carts (id, user_id, created_at, updated_at)
cart_items (id, cart_id, product_id, quantity, price, created_at, updated_at)

-- Đơn hàng
orders (id, user_id, total_amount, status, shipping_address, phone_number, created_at)
order_items (id, order_id, product_id, quantity, price)
```

```
-- Đánh giá
reviews (id, user_id, product_id, rating, comment, created_at, updated_at)

-- Yêu thích
wishlist (id, user_id, product_id, created_at)

-- Lịch sử chat AI
chat_history (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT NOT NULL,
    prompt TEXT NOT NULL,                    -- Câu hỏi của user
    response TEXT NOT NULL,                  -- Câu trả lời AI
    recommended_products TEXT,               -- IDs: "1,3,5,2"
    created_at TIMESTAMP
)
```

12. Frontend: Routing & State Management

Frontend xây dựng bằng **React 18 + Vite** với hệ thống routing bằng `react-router-dom`. Toàn bộ ứng dụng được bọc trong 2 Context Provider lồng nhau: `AuthProvider` (quản lý đăng nhập/đăng xuất) và `CartProvider` (quản lý giỏ hàng). Nhờ đó, **mọi component ở bất kỳ trang nào** đều có thể gọi `useAuth()` để biết user đang đăng nhập hay chưa, hoặc `useCart()` để thêm sản phẩm vào giỏ mà không cần props drilling. Trang nào cần đăng nhập sẽ tự kiểm tra `user` từ AuthContext — nếu null sẽ redirect về `/login`.

12.1. Routes

Ứng dụng có **12 routes** — 5 routes public (ai cũng truy cập được) và 7 routes yêu cầu đăng nhập:

URL	Component	Cần đăng nhập?	Mô tả
`/`	Home	Không	Trang chủ, hero banner, SP nổi bật
`/products`	ProductList	Không	Danh sách SP + tìm/lọc/sắp xếp
`/products/:id`	ProductDetail	Không	Chi tiết SP + reviews
`/login`	Login	Không	Form đăng nhập

URL	Component	Cần đăng nhập?	Mô tả
`/register`	Register	Không	Form đăng ký
`/cart`	Cart	Có	Giỏ hàng
`/checkout`	Checkout	Có	Thanh toán
`/order-success`	OrderSuccess	Có	Thông báo đặt hàng thành công
`/orders`	OrderHistory	Có	Lịch sử đơn hàng
`/profile`	Profile	Có	Hồ sơ cá nhân
`/wishlist`	Wishlist	Có	Danh sách yêu thích
`/ai-stylist`	AiStylist	Có	Chat AI tư vấn thời trang

12.2. Context Providers (Quản Lý State Toàn Cục)

Hai Context Provider hoạt động theo mô hình **"gọi API → cập nhật state → React tự re-render"**. Khi `AuthProvider` gọi `login()` , nó gửi request đến Backend, nhận JWT token, lưu vào `localStorage` (để persist qua reload), đồng thời set `user` state (để các component re-render ngay lập tức). Tương tự, `CartProvider` giữ cache giỏ hàng trong state — mỗi thao tác add/remove/update đều gọi API trước, sau đó cập nhật state local. Sơ đồ cấu trúc:

```
<App>
  └── <AuthProvider>                    ← Quản lý trạng thái đăng nhập
  │     State: { user }                   - user object (id, username, email, role)
  │     Methods:                          - login(credentials) → gọi API + set state
  │       login(), register(),            - register(data) → gọi API + set state
  │       logout()                        - logout() → clear localStorage + state
  │
  └─── <CartProvider>                 ← Quản lý giỏ hàng
  │       State: { cart, itemCount }      - cart object + tổng số items
  │       Methods:                        - addToCart() → gọi API + cập nhật state
  │         addToCart(), removeItem(),     - removeItem() → gọi API + cập nhật state
  │         updateQuantity(),             - clearCart() → sau khi đặt hàng thành công
  │         clearCart(), loadCart()
  │
  └──── <Routes>                      ← Tất cả pages có thể dùng useAuth()
          Mọi component con             và useCart() để truy cập state
          đều access được
          auth + cart context
```

12.3. Luồng Dữ Liệu trong axios (api.js)

File `api.js` là **điểm giao tiếp duy nhất** giữa Frontend và Backend. Tất cả 8 module API (`authAPI` , `productAPI` , `orderAPI` , `userAPI` , `wishlistAPI` , `reviewAPI` , `cartAPI` , `aiStylistAPI`) đều dùng chung một `axiosInstance` với base URL `http://localhost:8080/api` . Instance này có 2 interceptor:

- **Request interceptor:** Trước mỗi request, tự động lấy token từ `localStorage` và thêm vào header `Authorization: Bearer <token>` . Nếu token không tồn tại (chưa đăng nhập), header sẽ không được thêm.
- **Response interceptor:** Sau mỗi response, kiểm tra status code. Nếu 401 hoặc 403 (trừ API login/register), xóa `localStorage` và redirect về `/login` . Nếu lỗi khác, trả về error message cho component xử lý.

```
      Mọi API call
           │
┌──────────▼──────────┐
│ Request Interceptor │
│                     │
│ Tự động đính kèm    │
│ JWT token từ        │
│ localStorage vào    │
│ header Authorization│
└──────────┬──────────┘
           │
    Gửi request
           │
┌──────────▼──────────┐
│ Response Interceptor│
│                     │
│ Nếu 200 OK:         │
│   → Trả data        │
│                     │
│ Nếu 401/403:        │
│   → Xóa localStorage│
│   → Redirect /login │
│                     │
│ Nếu lỗi khác:       │
│   → Return error    │
│     message          │
└─────────────────────┘
```

13. Quy Trình Khởi Động Hệ Thống

Hệ thống cần khởi động **đúng thứ tự** vì có dependency giữa các service: Backend cần PostgreSQL đã chạy để kết nối database, và cần AI Service đã chạy để gọi API tư vấn (tuy nhiên Backend có fallback nên AI Service không bắt buộc). Frontend cần Backend đã chạy để gọi API. Thứ tự khuyến nghị: **Database → AI Service → Backend → Frontend**.

Lưu ý: Lần đầu khởi động AI Service sẽ mất 1-2 phút do phải tải model NLP (~500MB). Các lần sau sẽ nhanh hơn vì model đã được cache.

```
Bước 1: PostgreSQL (Docker)
    cd d:\Job\Minh
    docker-compose up -d db
    → Container: luxury-postgres
    → Port: 5432
    → DB: luxury_ecommerce (user: postgres, pass: postgres)
    → Tự chạy schema.sql để tạo bảng + seed data (2 users + 8 products)

Bước 2: AI Advisor Service (Python FastAPI)
    cd ai_advisor_service
    .\.venv\Scripts\python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8001
    → Load model NLP Sentence Transformers (~1-2 phút lần đầu)
    → Port: 8001
    → Health check: GET http://localhost:8001/health

Bước 3: Backend (Spring Boot + Maven)
    cd backend
    mvn spring-boot:run
    → Compile Java 17, kết nối PostgreSQL, JPA auto-migrate schema
    → Port: 8080
    → Test: GET http://localhost:8080/api/products

Bước 4: Frontend (Vite + React)
    cd d:\Job\Minh
    npm run dev
    → Port: 3000 (development)
    → Mở: http://localhost:3000

Bước 5 (Tuỳ chọn): pgAdmin — Quản lý Database
    docker-compose up -d pgadmin
    → Port: 5050
    → Login: admin@luxury.com / admin123
    → Kết nối DB: host.docker.internal:5432
```

14. Tài Khoản Mặc Định

File `schema.sql` tự động seed 2 tài khoản khi database được tạo lần đầu. Password được lưu dưới dạng **BCrypt hash** — không ai có thể đọc ngược password từ database. Tài khoản `admin` có role

ADMIN để quản lý hệ thống (thêm/sửa/xóa sản phẩm, xem tất cả đơn hàng, bật/tắt VIP cho user). Tài khoản `johndoe` là user thường để test chức năng mua hàng.

Username	Password	Role	Quyền hạn
`admin`	`admin123`	ADMIN	Quản lý sản phẩm, đơn hàng, users, chat AI
`johndoe`	`user123`	USER	Mua hàng, xem SP, chat AI, wishlist, review

15. Cấu Hình Quan Trọng

Dưới đây là các config chính cần biết khi deploy hoặc thay đổi môi trường. **JWT secret** là khoá bí mật dùng để ký token — nếu thay đổi, tất cả token cũ sẽ không còn hợp lệ (user phải đăng nhập lại). **CORS** chỉ cho phép frontend từ port 3000 và 5173 gọi API — nếu deploy lên domain khác cần cập nhật. **AI timeout** là 5 giây — nếu AI Service chưa trả lời trong 5s, Backend sẽ dùng fallback.

Backend (application.properties)

```
server.port=8080
spring.datasource.url=jdbc:postgresql://localhost:5432/luxury_ecommerce
spring.datasource.username=postgres
spring.datasource.password=postgres
jwt.secret=5367566B59703373367639792F423F4528482B4D6251655468576D5A71347437
jwt.expiration=86400000                    # 24 giờ
cors.allowed-origins=http://localhost:5173,http://localhost:3000
ai.python.base-url=http://localhost:8001
ai.python.timeout-ms=5000                 # Timeout 5s, sau đó dùng fallback
```

Frontend (api.js)

```
const API_BASE_URL = 'http://localhost:8080/api';
// Token lưu tại: localStorage.getItem('token')
// User info lưu tại: localStorage.getItem('user')
```

JWT Token Structure

```
Algorithm: HS512
Payload: { sub: "admin", iat: 1712345678, exp: 1712432078 }
Expiry: 24 giờ (86,400,000 ms)
```